PRODUCT REQUIREMENTS DOCUMENT (PRD)

- Purpose: Centralize municipal complaints from submission to verified resolution.
- Users:
    - Citizen: Submit complaints and track status.
    - Municipal Staff: Receive assignments and record work updates.
    - Administration: Manage departments/staff and review reports.
- Core Workflow: Submit complaint → Categorize → Assign department → Generate work order → Perform work → Update status → Verify resolution.
- Core Features: Complaint tracking, staff management, work assignments, inspections, status history, reports, role-based access, and operational alerts.
- Decision-Support Agent: Flag overdue complaints, workload imbalance, repeated issues, and recurring failures; require supervisor approval for consequential actions and log decisions.
- Success Criteria: Accurate records, traceable complaint status, valid database relationships, visible overdue issues, and reliable reports.
- Out of Scope: Payments, live GPS tracking, autonomous dispatch, external live integrations, and automated fraud accusations.
- Project Testing: Use synthetic municipal data.

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

- Technology Stack: Next.js frontend, Django REST Framework backend, PostgreSQL database.
- FR-01 — Citizens: Create and update citizen records.
- FR-02 — Complaints: Validate citizen details and complaint inputs; return correction messages for invalid data.
- FR-03 — Categorization: Categorize complaints and identify the responsible department.
- FR-04 — Registration: Save complaints and issue complaint numbers and acknowledgments.
- FR-05 — Work Assignment: Create work orders and assign staff to complaints.
- FR-06 — Status Updates: Record work performed, actions taken, completion details, and complaint status changes.
- FR-07 — Inquiry: Provide citizens with complaint status and resolution information.
- FR-08 — Reporting: Generate complaint reports, department performance summaries, and pending-work reports.
- FR-09 — Alerts: Generate explainable operational alerts; require human approval for consequential actions and record audit logs.
- Security: Enforce role-based permissions and protect personal data.
- Data Integrity: Enforce primary keys, foreign keys, validation, and transactions.
- Testing: Verify CRUD operations, workflow transitions, relationships, reports, and alert accuracy.

-

SYSTEM DESIGN SPECIFICATION (SDS)

- Architecture: Three-tier system: Next.js (UI), Django REST Framework (API/business logic), PostgreSQL (database).
- Database Entities:
    - CITIZEN: Citizen_ID (PK), Name, Phone, Address.
    - DEPARTMENT: Department_ID (PK), Department_Name, Office_Location.
    - STAFF: Staff_ID (PK), Department_ID (FK), Name, Phone, Designation.
    - COMPLAINT: Complaint_ID (PK), Citizen_ID (FK), Department_ID (FK), Complaint_Date, Category, Description, Location, Status.
    - WORK_ORDER: WorkOrder_ID (PK), Complaint_ID (FK), Staff_ID (FK), Assigned_Date, Completion_Date, Work_Status, Action_Taken.
- Relationships:
    - One Citizen submits many Complaints.
    - One Department handles many Complaints and employs many Staff.
    - One Complaint generates many Work Orders.
    - One Staff member performs many Work Orders.
- Data Integrity: Enforce PK/FK constraints, mandatory complaint ownership and department assignment, and transactional updates.
- Data Flow: External users interact with processes; processes access data stores. No direct external-entity-to-store or store-to-store flows.
- API Design: JSON requests/responses, consistent HTTP status codes, and server-side validation.
- Security: Enforce permissions server-side; protect credentials and secrets; use version-controlled database migrations.
- Decision-Support Agent: Apply transparent rules, explain alerts, require supervisor approval for consequential actions, and maintain audit logs.
- Testing: Verify database constraints, CRUD operations, status transitions, report accuracy, and agent explanations.